实验二：从时钟中断到四任务轮转——操作系统如何工作

真实姓名： 姚翔
学号： 20262822
整理日期： 2026 年 10 月 11 日

原创声明： 本报告的分析与整理署名姚翔；课程示例代码保留原作者 Mengning 的版权说明。原创作品转载请注明出处。
课程来源： 《Linux内核分析》MOOC。

1. 实验环境与步骤

实验使用实验楼提供的 Linux 虚拟机，内核源码版本为 Linux 3.9.4。进入源码目录，应用 mykernel 补丁并编译、启动内核：

```
cd ~/LinuxKernel/linux-3.9.4
rm -rf mykernel
patch -p1 < ../mykernel_for_linux3.9.4sc.patch
make allnoconfig
make
qemu -kernel arch/x86/boot/bzImage
```

内核编译完成后，使用 QEMU 启动生成的 `bzImage`。运行时，主循环输出与时钟中断输出交替出现，实验结果如下。

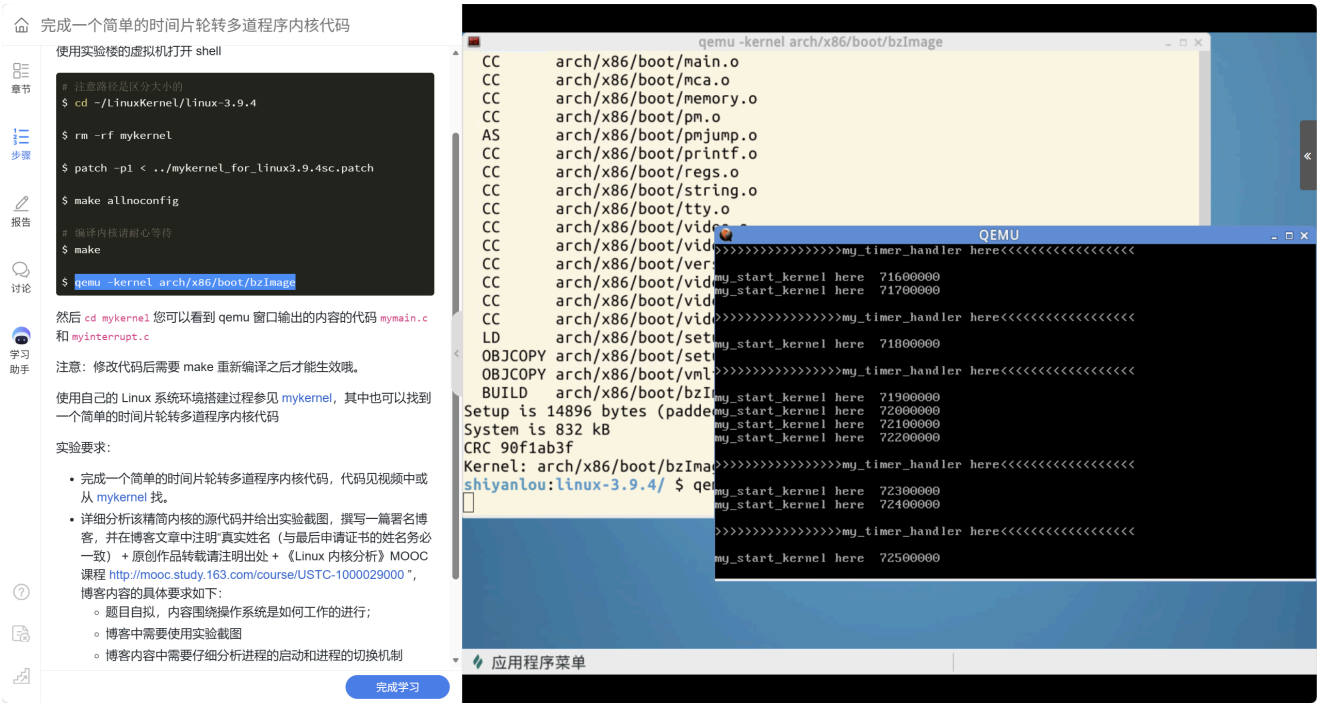


图1： QEMU 中 `my_start_kernel here` 与 `my_timer_handler here` 交替出现。

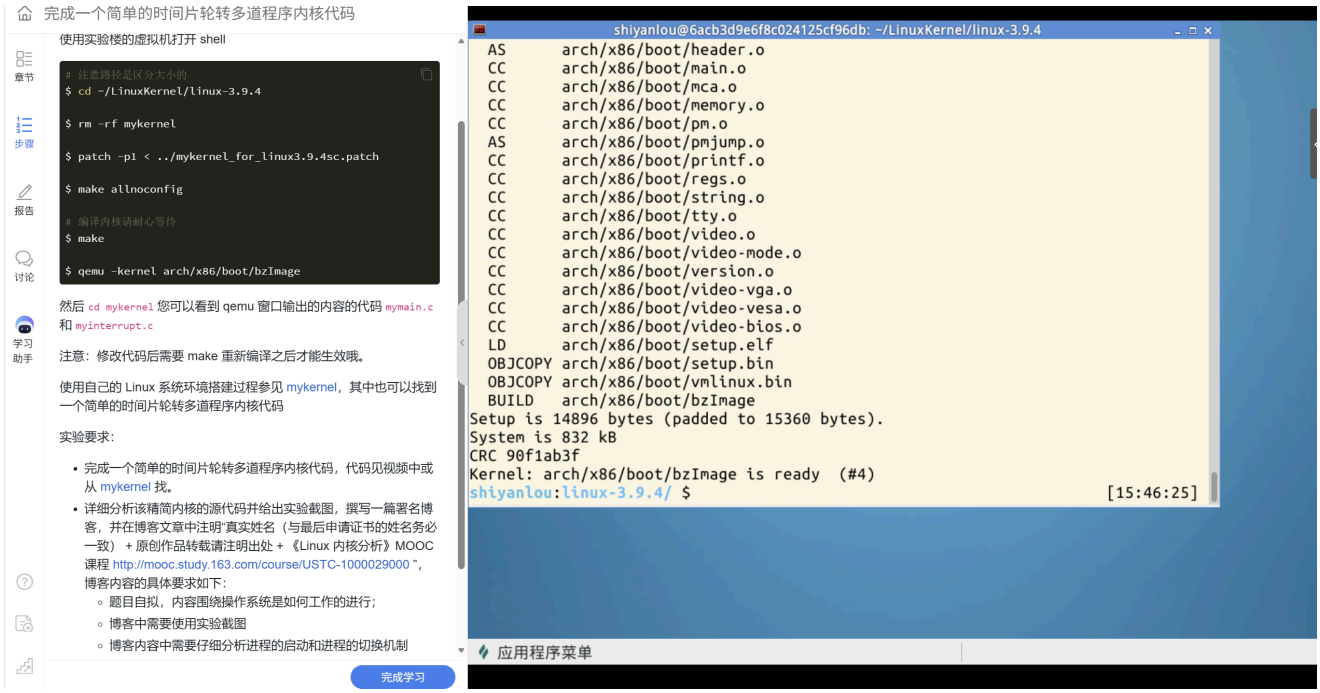


图2： 编译完成，生成 `arch/x86/boot/bzImage`。

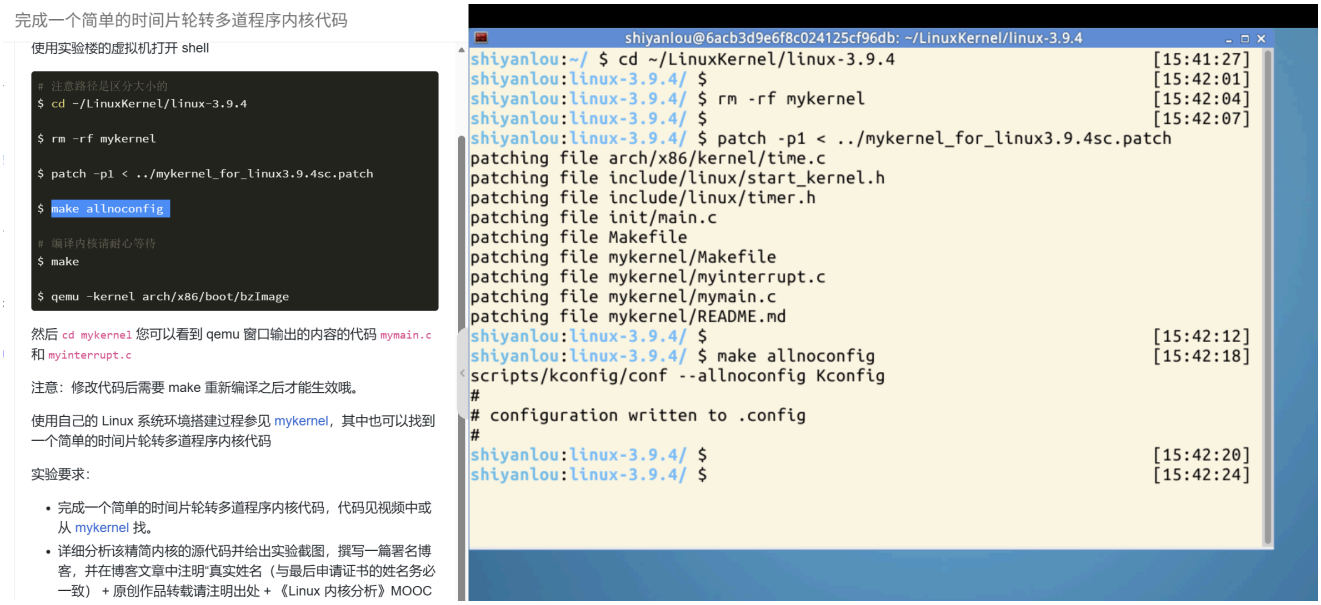


图3： mykernel 补丁应用完成，配置写入 `.config`。

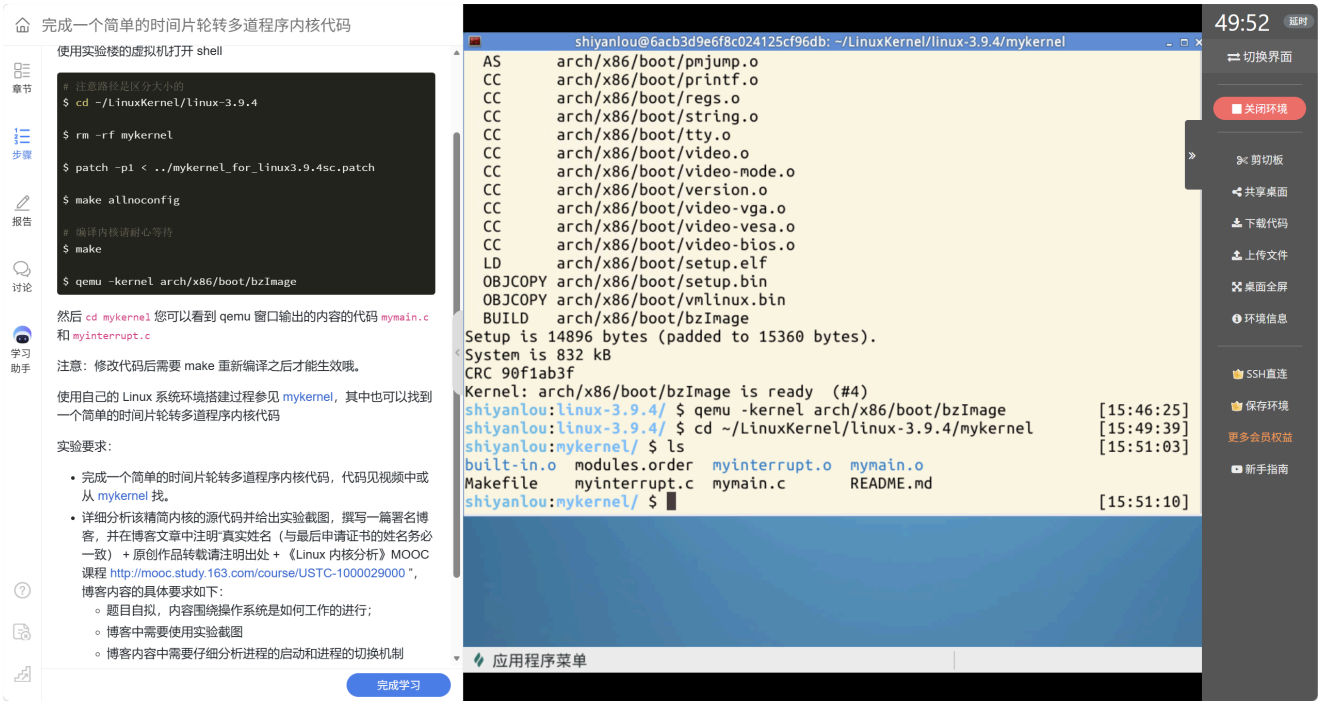


图4： mykernel 目录中的源文件及构建产物。

2. 操作结果判读

（1）应用补丁并生成配置

patch -p1 中的 -p1 去掉补丁路径最前面一级目录；当前目录应是内核源码根目录。make allnoconfig 生成尽量精简的配置，具体生效选项仍受 Kconfig 依赖和默认规则影响，不能理解成“最后得到一个完全没有任何功能的内核”。

结果判读： 终端列出 arch/x86/kernel/time.c、init/main.c、mykernel/mymain.c、mykernel/myinterrupt.c 等补丁目标，随后显示 configuration written to .config。这说明补丁不仅增加两个教学文件，还修改了启动与时钟路径，把教学代码接入 Linux 提供的运行环境。

（2）检查内核编译的完成标志

结果判读： 末尾出现 BUILD arch/x86/boot/bzImage，并明确显示：

```
Setup is 14896 bytes (padded to 15360 bytes).
System is 832 kB
CRC 90f1ab3f
Kernel: arch/x86/boot/bzImage is ready (#4)
```

判断编译成功应看最终镜像生成提示，而不能只凭中途有很多 CC、AS 或 LD 输出。它们分别表示编译、汇编和链接过程；图2的最终结果证明此次生成了可交给 QEMU 的内核镜像。

（3）在 QEMU 中观察主循环和时钟中断

结果判读： QEMU 窗口中交替出现 my_start_kernel here 和 my_timer_handler here。主循环计数从 7160000、7170000 等继续增加，中断信息插入其间，随后主循环继续输出。

这表明基础教学入口已经运行，周期性时钟中断也能得到处理。它还说明无限循环不会阻止硬件中断带来的控制流转移。轮转版本在这一基础上增加 PCB、调度标志和上下文切换，后面按源码详细展开。

（4）找到与输出对应的源码

```
cd ~/LinuxKernel/linux-3.9.4/mykernel
ls
```

结果判读： 目录中能看到 `mymain.c`、`myinterrupt.c`、`Makefile`、`README.md`，以及 `mymain.o`、`myinterrupt.o`、`built-in.o` 等构建产物。源码文件与目标文件不是同一种材料；修改 `.c` 后必须回到内核源码根目录重新 `make`，再启动更新后的 `bzImage`。

基础补丁中的两个 C 文件提供入口和中断环境；轮转模型进一步增加 `mypcb.h`，把这两个执行路径连接到任务管理与调度机制。

3. 轮转示例源码与整体结构

下面按文件给出课程 IA-32 示例的学习整理版，结合前面的实验结果建立整体结构，随后逐段分析。它用于解释实验要求中的轮转内核。代码适用于课程 Linux 3.9.4 的 32 位教学环境，不能直接混入教材 Linux 5.4.34 的 64 位示例。

来源固定为 [mengning/mykernel 的历史版本 726a79f]，避免上游现版升级为 64 位后出现混用。为使控制流分析完整，我在 `myinterrupt.c` 的首次启动分支中补全了向前引用 `1f` 对应的续接标签 `1:` 与 `popl %%ebp`；其余保留示例结构，仅统一排版。下文围绕这一整理版追踪任务创建、首次启动和上下文恢复。

（1）mypcb.h：描述任务及保存运行位置

```
/*
 *  linux/mykernel/mypcb.h
 *
 *  Kernel internal PCB types
 *
 *  Copyright (C) 2013  Mengning
 *
 */

#define MAX_TASK_NUM        4
#define KERNEL_STACK_SIZE   1024*8

/* CPU-specific state of this task */
struct Thread {
    unsigned long        ip;
    unsigned long        sp;
};

typedef struct PCB{
    int pid;
    volatile long state;    /* -1 unrunnable, 0 runnable, >0 stopped */
    char stack[KERNEL_STACK_SIZE];
    /* CPU-specific state of this task */
    struct Thread thread;
```

```
    unsigned long    task_entry;
    struct PCB *next;
}tPCB;

void my_schedule(void);
```

（2）mymain.c：建立任务链表并启动第一个任务

```
/*
 *  linux/mykernel/mymain.c
 *
 *  Kernel internal my_start_kernel
 *
 *  Copyright (C) 2013  Mengning
 *
 */
#include <linux/types.h>
#include <linux/string.h>
#include <linux/ctype.h>
#include <linux/tty.h>
#include <linux/vmalloc.h>


#include "mypcb.h"

tPCB task[MAX_TASK_NUM];
tPCB * my_current_task = NULL;
volatile int my_need_sched = 0;

void my_process(void);


void __init my_start_kernel(void)
{
    int pid = 0;
    int i;
    /* Initialize process 0*/
    task[pid].pid = pid;
    task[pid].state = 0;/* -1 unrunnable, 0 runnable, >0 stopped */
    task[pid].task_entry = task[pid].thread.ip = (unsigned long)my_process;
    task[pid].thread.sp = (unsigned long)&task[pid].stack[KERNEL_STACK_SIZE-1];
    task[pid].next = &task[pid];
    /*fork more process */
    for(i=1;i<MAX_TASK_NUM;i++)
    {
        memcpy(&task[i],&task[0],sizeof(tPCB));
        task[i].pid = i;
        task[i].state = -1;
        task[i].thread.sp = (unsigned long)&task[i].stack[KERNEL_STACK_SIZE-1];
        task[i].next = task[i-1].next;
        task[i-1].next = &task[i];
    }
    /* start process 0 by task[0] */
    pid = 0;
    my_current_task = &task[pid];
    asm volatile(
        "movl %1,%%esp\n\t"     /* set task[pid].thread.sp to esp */
        "pushl %1\n\t"            /* push ebp */
        "pushl %0\n\t"            /* push task[pid].thread.ip */
        "ret\n\t"                 /* pop task[pid].thread.ip to eip */
        "popl %%ebp\n\t"
        :
```

```
        : "c" (task[pid].thread.ip),"d" (task[pid].thread.sp)    /* input c or d mean %ecx/%edx*/
    );
}
void my_process(void)
{
    int i = 0;
    while(1)
    {
        i++;
        if(i%10000000 == 0)
        {
            printk(KERN_NOTICE "this is process %d -\n",my_current_task->pid);
            if(my_need_sched == 1)
            {
                my_need_sched = 0;
                my_schedule();
            }
            printk(KERN_NOTICE "this is process %d +\n",my_current_task->pid);
        }
    }
}
```

（3）myinterrupt.c：提出调度请求并切换任务

```
/*
 *  linux/mykernel/myinterrupt.c
 *
 *  Kernel internal my_timer_handler
 *
 *  Copyright (C) 2013  Mengning
 *
 */
#include <linux/types.h>
#include <linux/string.h>
#include <linux/ctype.h>
#include <linux/tty.h>
#include <linux/vmalloc.h>

#include "mypcb.h"

extern tPCB task[MAX_TASK_NUM];
extern tPCB * my_current_task;
extern volatile int my_need_sched;
volatile int time_count = 0;

/*
 * Called by timer interrupt.
 * it runs in the name of current running process,
 * so it use kernel stack of current running process
 */
void my_timer_handler(void)
{
#if 1
    if(time_count%1000 == 0 && my_need_sched != 1)
    {
        printk(KERN_NOTICE ">>>my_timer_handler here<<<\n");
        my_need_sched = 1;
    }
    time_count ++ ;
#endif
    return;
}
```

```
void my_schedule(void)
{
    tPCB * next;
    tPCB * prev;

    if(my_current_task == NULL
        || my_current_task->next == NULL)
    {
        return;
    }
    printk(KERN_NOTICE ">>>my_schedule<<<\n");
    /* schedule */
    next = my_current_task->next;
    prev = my_current_task;
    if(next->state == 0)/* -1 unrunnable, 0 runnable, >0 stopped */
    {
        my_current_task = next;
        printk(KERN_NOTICE ">>>switch %d to %d<<<\n",prev->pid,next->pid);
        /* switch to next process */
        asm volatile(
            "pushl %%ebp\n\t"         /* save ebp */
            "movl %%esp,%0\n\t"     /* save esp */
            "movl %2,%%esp\n\t"     /* restore  esp */
            "movl $1f,%1\n\t"       /* save eip */
            "pushl %3\n\t"
            "ret\n\t"                 /* restore  eip */
            "1:\t"                  /* next process start here */
            "popl %%ebp\n\t"
            : "=m" (prev->thread.sp),"=m" (prev->thread.ip)
            : "m" (next->thread.sp),"m" (next->thread.ip)
        );

    }
    else
    {
        next->state = 0;
        my_current_task = next;
        printk(KERN_NOTICE ">>>switch %d to %d<<<\n",prev->pid,next->pid);
        /* switch to new process */
        asm volatile(
            "pushl %%ebp\n\t"         /* save ebp */
            "movl %%esp,%0\n\t"     /* save esp */
            "movl %2,%%esp\n\t"     /* restore  esp */
            "movl %2,%%ebp\n\t"     /* restore  ebp */
            "movl $1f,%1\n\t"       /* save eip */
            "pushl %3\n\t"
            "ret\n\t"                 /* restore  eip */
            "1:\n\t"                 /* Resume prev here when scheduled again. */
            "popl %%ebp\n\t"          /* Restore the saved frame pointer. */
            : "=m" (prev->thread.sp),"=m" (prev->thread.ip)
            : "m" (next->thread.sp),"m" (next->thread.ip)
        );
    }
    return;
}
```

4. 代码分析：PCB 与任务上下文

三个文件的关系可以概括为：`mypcb.h` 定义“保存什么”，`mymain.c` 决定“任务从哪里开始”，`myinterrupt.c` 处理“何时请求切换、如何保存和恢复”。上面的 `Makefile` 由实验补丁提供，头文件无需单独编译。

tPCB 是教学进程控制块。各字段的用途如下：

字段	含义	在切换中的作用
pid	任务编号	用于识别输出来自哪个任务
state	示例的简化状态	区分已有运行现场和尚待首次启动的任务
stack[8192]	每个任务的独立栈区域	保存局部变量、调用链和切换现场
thread.sp	保存的栈指针	找到该任务的栈现场
thread.ip	保存的执行位置	首次为入口，运行后为续接位置
task_entry	任务入口地址	说明任务最初应从哪里开始
next	下一个 PCB	构成轮转链表

四个任务都执行 my_process()，代码地址相同，但栈和保存的运行位置不同，所以它们不是把同一个循环简单打印四次。任务身份由当前 PCB 和相应上下文共同决定。

这里的 state=-1 被示例用作首次启动的标记，调度器的 else 分支会把它置为 0 并启动。它不能直接等同于完整 Linux 的阻塞状态；真实 Linux 中“尚未启动”“睡眠”“可运行”等关系更复杂。同样，memcpy 复制 PCB 只是在教学模型中建立任务，不能称为实现了完整的 fork()。

5. 任务创建：为什么要重新设置每个任务的栈指针

0 号任务先设置 PID、运行状态、入口和栈指针，next 最初指向自己。随后每次复制 task[0]，都必须重写新任务的 PID、状态、栈指针和链表指针。

最容易漏掉的是：

```
task[i].thread.sp =
    (unsigned long)&task[i].stack[KERNEL_STACK_SIZE-1];
```

如果只复制 PCB 而不重新赋值，多个任务保存的栈指针仍可能指向 0 号任务的栈，局部变量和返回地址就会互相覆盖。复制结构体并不自动把其中的指针“变成新对象内部对应位置的指针”。

循环插入完毕后得到：

```
task[0] -> task[1] -> task[2] -> task[3]
   ^                              |
   |------------------------------|
```

因为最后一个任务的 next 指回 0 号任务，调度器反复取 current->next，才能实现 0 -> 1 -> 2 -> 3 -> 0 的循环顺序。

6. 首次启动：不是只给 my_current_task 赋值

my_current_task = &task[0] 只更新软件中的当前任务指针，CPU 还没有因此自动使用 0 号任务的栈。首次运行必须设置实际的 ESP，并把入口地址送入控制流。

在启动汇编中，`movl %1,%%esp` 切换到任务栈；`pushl %0` 把入口地址压栈；`ret` 将栈顶地址弹出作为 EIP，因而进入 `my_process()`。这里使用 `ret` 是借助其“从栈顶取得下一执行地址”的效果，不表示之前一定发生过普通的函数调用。

还需要注意源码中 `ret` 后的 `popl %%ebp`：首次跳入无限循环的 `my_process()` 后，不会顺序执行这条指令。不能把它当作“首次启动立刻恢复 EBP”的步骤。该教学代码依赖指定的编译环境和栈组织；理解这个跳转性质，比逐行机械翻译注释更重要。

7. 时钟中断：提出调度请求与实际切换是两件事

`my_timer_handler()` 在计数满足条件且没有待处理请求时，把 `my_need_sched` 设为 1。`my_process()` 在循环检查点发现此标志后清零，再调用 `my_schedule()`。

```
时钟中断
  -> my_timer_handler()
  -> my_need_sched = 1
  -> 从中断返回
  -> 当前任务继续到检查点
  -> my_schedule()
```

因此，这个模型由时钟事件推动轮转，但切换发生在任务主动检查标志的位置。不能把它写成“每次时钟中断都直接切到下一个任务”，也不能说它已经实现了任意指令处的完整抢占式调度。

另一个细节是 `time_count` 初值为 0，第一次检查 `time_count % 1000 == 0` 就成立，并不是先累计 1000 次才提出首次请求。1000 表示计数间隔；如果没有确认时钟频率，就不能把它直接写成 1000 毫秒。忙循环的 `i % 10000000` 也是检查间隔，不是可靠的时间计量。

8. 上下文切换：保存旧栈，恢复新栈，再恢复执行位置

调度器先取 `prev=current` 和 `next=current->next`，在切换前把 `my_current_task` 更新为 `next`。否则 CPU 已经在新任务栈中运行，而日志仍可能用旧任务的 PID，出现“运行现场和当前任务指针不一致”的问题。

对于已经运行过的任务，关键步骤是：

汇编操作	作用
`pushl %%ebp`	把旧任务的帧指针保存在旧栈
`movl %%esp,%0`	把旧 ESP 写入旧 PCB
`movl %2,%%esp`	改用新任务保存的 ESP
`movl $1f,%1`	保存旧任务将来恢复的续接地址
`pushl %3; ret`	进入新任务保存的执行位置
`1:; popl %%ebp`	此任务将来恢复时取回自己的 EBP

`1f` 是汇编局部标签的向前引用，表示后面的 `1:`。保存的 `thread.ip` 不是把 EIP 当作普通寄存器直接搬出，而是记录一个约定的恢复点。

切换后，原任务并没有正常顺序执行完当前 `my_schedule()` 调用，而是暂时挂起。当轮转回这个任务时，恢复它原来的栈并跳到保存的标签，再沿原调用链返回 `my_process()`。局部变量和返回地址之所以能继续使用，关键在于任务各有独立栈。

首次启动与恢复旧任务的区别： 新任务尚未留下调用栈和续接地址，只能从 `my_process` 入口开始；旧任务已在切换时保存了现场，必须从自己的续接位置恢复。把两种情况都跳回入口，会导致旧任务的局部状态重新初始化，失去“继续执行”的意义。

前面学习整理版对首次启动分支补全续接标签，就是为了让“旧任务在该分支被切出后，未来从哪里回来”有明确答案。这样才能让首次启动分支保存的地址与日后的恢复位置配对。

9. 一次完整轮转的预期控制流

以下按课程源码推导一次完整轮转：

```
task 0 运行
  -> 时钟提出调度请求
  -> task 0 到达检查点并调用 my_schedule
  -> 保存 task 0 的栈和续接位置
  -> 首次启动 task 1
  -> 同理启动 task 2、task 3
  -> 恢复 task 0 先前保存的上下文
  -> task 0 从暂停的位置继续运行
```

若完成轮转版本的运行，应同时观察任务编号变化、`my_schedule` 和 `switch` 输出，并检查最后能否从 3 回到 0。光看到任务 1 的入口打印，还不足以证明“旧任务的恢复”正确。

源代码里的 `this is process N -` 在检查点前打印，`this is process N +` 在调度调用返回后打印。任务被切出后，它的 `+` 可能要等下一轮恢复才出现，不要求一对符号在输出中紧邻。日志的顺序要结合保存和恢复位置分析，不能只按屏幕行距判断。

10. 两个关键切换案例：0→1 与 3→0

（1）0→1：给一个新任务建立第一次执行机会

调度前，0 号任务正在自己的 `my_process()` 中运行，1 号任务的 `state=-1`，`thread.ip` 仍是 `my_process` 入口，`thread.sp` 指向其独立栈的初始位置。

1. 0 号任务在检查点读到 `my_need_sched=1`，清零后调用 `my_schedule()`。
2. 调度器得到 `prev=&task[0]`、`next=&task[1]`，进入首次启动分支。
3. 设置 1 号任务状态为 0，并把当前任务指针更新为 1。
4. 在 0 号栈压入 EBP，保存这时的 ESP 和未来的恢复标签。
5. ESP 改为 1 号任务的栈，EBP 也按教学代码初始化为该栈指针。
6. 将 1 号任务入口压栈，`ret` 弹出该地址，CPU 从 `my_process()` 开始运行。
7. 1 号任务建立自己的函数栈帧和局部变量 `i`；0 号任务的调用链留在原栈中。

用于理解的输出顺序是：

```
this is process 0 -
>>>my_schedule<<<
>>>switch 0 to 1<<<
this is process 1 -
```

这是理论示意，省略了其他循环打印和时钟消息。调度请求可能早已被中断设置，所以 `my_timer_handler` 的打印不必紧邻 `my_schedule`。0 号任务的 `+` 暂时未出现，是因为它还没有从自己的调度调用返回。

（2）3→0：不是重新创建 0 号任务，而是恢复它

当 1、2、3 号任务依次得到首次启动机会后，3 号任务的 `next` 指向 0。此时 0 的 `state=0`，且 PCB 中已经保存了上次被切出时的栈指针和续接标签。

1. 3 号任务到达检查点，进入调度器。
2. 调度器确定 `prev=&task[3]`、`next=&task[0]`，进入恢复分支。
3. 保存 3 号任务的 EBP、ESP 和恢复地址，以备下一轮使用。
4. ESP 改为 0 号任务曾经保存的值。
5. 把 0 号任务的 `thread.ip` 压入 0 号栈，再用 `ret` 进入它自己的续接点。
6. 0 号任务在上次切出分支的 `1:` 后恢复 EBP，随后完成原来的 `my_schedule()` 调用。
7. CPU 回到 0 号任务 `my_process()` 中的调度调用之后，打印相应的 `+`，然后继续循环。

理论示意如下：

```
this is process 3 -
>>>my_schedule<<<
>>>switch 3 to 0<<<
this is process 0 +
```

这里出现 `process 0 +`，恰好说明应从“旧调用的返回位置”继续，而不是再次执行 `my_process()` 的初始化。0 号任务的局部计数仍由其原栈承载；如果每轮都回到入口，计数会被重置，任务也无法保留此前的执行进度。

（3）从寄存器、PCB 和栈三方面检查切换

阶段	CPU 使用的栈	PCB 中保存的内容	控制流位置
0 正在运行	task 0 栈	1 的入口和初始 SP 已准备	0 的 `my_process`
保存 0	task 0 栈	0 的 SP 与续接地址写回	0 的 `my_schedule`
首次启动 1	task 1 栈	当前任务指针为 1	1 的 `my_process` 入口
3 准备恢复 0	先使用 task 3 栈	保存 3，读取 0 的旧现场	3 的 `my_schedule`
0 恢复	task 0 原栈	0 的旧现场已用于恢复	0 的续接标签，然后返回原循环

需要保存的现场不仅是“下一条指令”。如果只恢复执行地址而仍使用别人的栈，函数返回和局部变量都会失去对应关系；如果只换栈却跳回入口，也无法继续原来的调用链。栈位置与控制流位置必须配套。

11. 实验中的问题、解决方法与验证依据

下面归纳实验操作和源码学习中需要解决的问题，给出原因、解决方法和判断依据。

问题或疑问	原因分析	解决方法	依据与验证范围
怎样判断补丁是否进入正确目录?	补丁的相对路径依赖当前目录与 `-p1`	先进入 `linux-3.9.4`，核对补丁目标	图3列出启动、时钟及 mykernel 文件
怎样判断编译真正结束?	中途编译输出不代表最终镜像可用	检查 `bzImage is ready` 与输出路径	图2已验证镜像生成
无限循环为何还会打印中断消息?	硬件中断能暂时改变 CPU 控制流	分开追踪主循环与中断处理入口	图1已验证交替输出
找到源文件后，修改为什么需要重新编译?	QEMU 运行的是镜像，而不是磁盘上的 C 文本	回源码根目录 `make`，再启动新镜像	图4用于定位源文件与构建产物
为什么要为每个任务重写 SP?	PCB 复制不会自动修正内部地址	SP 指向该任务自己的栈	轮转初始化源码分析
中断出现是否就是任务切换?	中断只设置标志，任务检查点才调用调度器	跟踪 `my_need_sched` 的写入与检查	轮转源码分析
为何恢复后不能再从入口开始?	旧任务已有未完成调用链和局部状态	保存续接地址和独立栈，成对恢复	汇编与栈布局推导
历史代码的 `1f` 在首次启动分支如何续接?	被切出任务需要对应的恢复标签	学习整理版补全 `1:` 与 EBP 恢复	汇编标签与栈恢复的对应关系
能否直接复制上游最新代码?	新版可能已改成 x86-64，寄存器与指令宽度不同	固定 IA-32 历史版本，并说明教材版本	上游版本记录与截图环境

12. 我对“操作系统如何工作”的理解

CPU 始终是在执行具体指令。操作系统要管理任务，也必须先获得执行机会。时钟中断提供了周期性进入内核代码的机会，PCB 则提供了记录各任务身份、栈和执行位置的结构。

调度策略决定下一个任务是谁，上下文切换落实“把 CPU 的运行现场改成另一个任务的现场”。循环链表可以给出简单轮转顺序，但如果没有独立栈和可靠的恢复地址，轮转策略就无法成为可持续运行的多任务机制。

本次实践直接验证了基础主循环和中断环境。结合课程源码，我进一步理解了从“中断发生”到“提出调度请求”，再到“保存旧任务、启动或恢复新任务”的完整关系。四任务共享同一段代码的模型适合看清这个核心机制；真实 Linux 还需要地址空间、权限、完整寄存器现场、同步与复杂调度策略，不能用这百余行教学代码概括全部实现。

参考资料

1. 《Linux内核分析》MOOC。
2. mykernel 32 位历史源码与实验说明，原作者 Mengning。